

当代人工智能实验课：实验一 - 文本分类

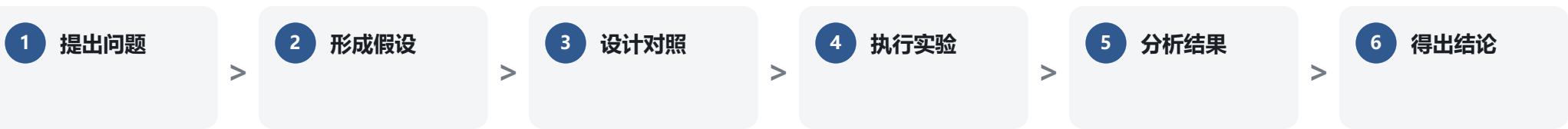
助教: 余婧 (51285903044@stu.ecnu.edu.cn) 张珈鸣 (51285903058@stu.ecnu.edu.cn)

日期: 2025年9月17日

提交截止日期: 2025年10月15日23:59:59

提交方式后续在班级群里公布

实验不是反复运行代码直到得到理想数字，而是在可控制、可比较的条件下检验一个问题或假设。



基本原则

- 一次实验尽量只改变需要研究的因素，其他条件保持一致。
- 先建立简单、可运行的基线，再判断新增方法是否真正带来改进。
- 实验前确定评价指标和比较方式，避免根据结果临时改变标准。
- 完整记录数据、配置、运行过程和失败结果，使结论能够被复查与复现。

三个数据子集承担不同职责；分开使用是为了获得可信的泛化能力估计。

数据集	主要作用	使用原则
训练集	学习模型参数和数据规律	可用于拟合；预处理统计量也只能从这里学习
验证集	选择方案、超参数与停止时机	用于调整超参反复比较，但不能参与模型拟合
测试集	在方案确定后进行最终评估	应保持封存；不能用于调参或挑选模型

如何划分

- 划分应尽量保持各子集具有代表性；比例没有唯一答案，应结合数据规模决定。
- 存在类别、个体、群组或时间依赖时，应采用与数据结构相匹配的分层、分组或时间顺序划分。

数据泄露： 任何让验证集或测试集信息提前影响训练和选择过程的行为，都会使评估结果虚高。

正确顺序：先划分数据，再仅用训练集确定处理规则，最后把同一规则应用于验证集和测试集。

数据预处理

- 目标是提高数据质量、统一表示形式，并满足算法的输入要求。
- 常见操作包括缺失值处理、异常值检查、编码、缩放、归一化和特征构造。
- 涉及均值、方差、词表或其他统计量时，只能在训练集上拟合。
- 保留处理前后的检查结果，确认没有误删样本、改变标签或引入偏差。

数据增强

- 通过合理变换生成新训练样本，目的是提升鲁棒性和泛化能力。
- 增强方法必须符合数据语义，并保证标签在变换后仍然成立。
- 通常只增强训练集；验证集和测试集应保持稳定，以便公平比较。
- 增强并非越多越好，应与未增强的基线对比并检查分布变化。

常见误区：在划分前对全部数据去重、统计、归一化或增强，可能把本应不可见的信息带入训练过程。

参数由模型从训练数据中学习；超参数由实验者预先设定。调参依据验证结果，而不是测试结果。

推荐流程

1 建立基线

先用合理默认值跑通流程。

2 确定范围

根据含义设置候选范围。

3 选择策略

可用手动、网格或随机搜索。
一般设置等比或者等差数列

4 公平比较

固定数据、指标和运行条件。

5 锁定方案

验证完成后再做最终测试。

从现象判断下一步

训练和验证都较差

可能欠拟合：检查数据表示、模型能力、
训练是否充分。

训练好、验证明显较差

可能过拟合：考虑正则化、简化模型、
增加有效数据。

结果波动很大

检查样本量、随机性和划分敏感性，
必要时重复运行。

如何评价实验

- 指标应与任务目标一致；必要时同时报告多个互补指标。
- 除总体数字外，还应观察不同类别、区间或群体的表现。
- 通过错误分析、对照实验和消融实验解释结果来自哪里。
- 存在随机性时重复运行，并报告均值、波动或不确定性。

如何保证可复现

- 固定并记录随机种子，同时说明仍可能存在的非确定性。
- 记录数据版本、软件依赖、硬件环境和所有实验配置。
- 保存代码、配置、日志和必要的中间产物，避免依赖绝对路径。
- 从干净环境按说明重新运行一次，确认结果可以独立得到。

实验报告应形成完整证据链

✓ 问题、假设和评价标准是否清楚

✓ 基线、变量控制和调参过程是否可追溯

✓ 结论是否与证据一致，并说明局限性

✓ 数据来源、划分和处理过程是否说明

✓ 结果、误差分析和不确定性是否完整

核心原则：结果可以不理想，但实验过程必须诚实、清晰、可比较、可复现。

大家好！欢迎来到我们的第一个实验：文本分类。本次实验的目标是：

1. 理解文本分类的基本概念和流程。
2. 学习并实践文本预处理和特征提取的关键技术，如TF-IDF。
3. 掌握几种经典的机器学习分类算法，并学会如何使用Python库来调用它们。
4. 学习如何评估和比较不同分类模型的性能。

在开始之前，请确保你已经安装好了python，并且推荐安装conda用于不同实验的环境管理，可以自行搜索相关教程。

在完成python以及conda的安装之后，请按照以下命令启动并且安装必要的python库：

```
conda create -n project1 python=3.10    # project1可以换成任意命名， python版本建议不低于 python3.7.  
conda activate project1                 # 如果你更改了上一行 project1的名字，这里要同步更改。  
conda install numpy pandas scikit-learn # 注意， scikit-learn在安装时和导入时名称不一样
```

文本分类，顾名思义，就是将一段文本自动地归入预先定义好的类别中。这是一个典型的监督学习问题。例如，将新闻文章分为“体育”、“科技”、“财经”等类别；或将用户评论分为“正面”、“负面”、“中性”。

一个完整的文本分类（或者其他任何深度学习的项目）流程通常包括以下步骤：

1. 数据加载、划分与探索：读取数据集，了解数据内容和结构。
2. 文本预处理：清洗文本数据，例如去除停用词、标点符号等。
3. 特征工程：将文本数据转换为机器学习模型可以理解的数值形式（向量）。最经典的方法之一是 TF-IDF。
4. 模型训练：选择一个分类算法，在处理好的数据上进行训练。
5. 模型评估：使用测试集来评估模型的性能，并对不同模型进行比较。
6. 模型预测：使用训练好的模型对新的、未知的文本进行分类。

第二部分：数据集介绍与加载

实验一 / Experiment 1

我们选用一个新闻文本10分类数据集。为了大家实验更方便、提交结果更统一、性能评估更公平，我们提前划分了训练集以及测试集，并且在draft_main.py中为大家实现了加载数据的部分。但是我们并没有为大家划分验证集，这一步需要大家自己设置比例，完成验证集的划分。

但是注意，这个加载数据的函数仅仅是读取了数据，并不会进行任何的数据清洗以及数据增强。我们欢迎大家自行探索并尝试数据清洗，这里可以考虑的数据清洗的方法包括但不限于：去除停用词、去除正文以外的信息、去除标点符号、对单词进行stemming。

data-Project1：包含 draft_main.py、TUNING.md、训练/测试 CSV、predictions.csv 示例文件。

计算机无法直接理解文本，我们需要将文本转换成数值向量。TF-IDF (Term Frequency-Inverse Document Frequency) 是一种非常有效的文本表示方法。

- TF (词频): 一个词在文档中出现得越频繁，它对于这篇文档就越重要。
- IDF (逆文档频率): 一个词在越多的文档中出现，它的区分能力就越弱，重要性就越低。

我们将使用 scikit-learn 的 TfidfVectorizer 来完成这个转换过程。它会自动完成一部分文本预处理（如分词、转小写）和TF-IDF计算。draft_main.py中已经为大家搭建了这一步的框架，可以自行探索相关的参数设置。

现在我们有了向量化的数据，可以开始训练分类模型了。我们将介绍几种经典的算法。

算法一：朴素贝叶斯 (Naive Bayes)

这是文本分类中最基础、最常用的机器学习算法之一，它基于贝叶斯定理，并假设特征之间相互独立。其计算简单，效率高。

我们为大家提供一段参考代码：

```
from sklearn.naive_bayes import MultinomialNB

# 1. 创建模型实例
nb_classifier = MultinomialNB()

# 2. 训练模型
print("\n--- 开始训练朴素贝叶斯模型 ---")
nb_classifier.fit(X_train_tfidf, y_train)
print("模型训练完成! ")

# 3. 进行预测
y_pred_nb = nb_classifier.predict(X_test_tfidf)
```

算法二：支持向量机 (SVM)

支持向量机 (Support Vector Machine, SVM) 是一种非常强大的分类器。

其核心思想是在特征空间中找到一个能将不同类别样本最大间隔地分开的超平面。对于线性不可分的数据，SVM可以使用核技巧 (Kernel Trick) 将其映射到高维空间，从而实现非线性分类。

scikit-learn 中的 SVC (Support Vector Classifier) 提供了SVM的实现。同样，我们也为大家提供核心参考代码：

```
from sklearn.svm import SVC

# 1. 创建模型实例
# C是正则化参数, kernel='linear'表示我们使用线性核
# 对于文本数据, 线性核通常表现得非常好且速度快
svm_classifier = SVC(kernel='linear', C=1.0, random_state=42)

# 2. 训练模型
print("\n--- 开始训练支持向量机 (SVM) 模型 ---")
svm_classifier.fit(X_train_tfidf, y_train)
print("模型训练完成! ")

# 3. 进行预测
y_pred_svm = svm_classifier.predict(X_test_tfidf)
```

Tips: 在初始化SVC时，可以尝试不同的 kernel (如 'rbf') 和调整参数 C 与 gamma，观察对模型性能的影响。

算法三：逻辑回归 (Logistic Regression)

虽然名字里有“回归”，但逻辑回归是工业界应用最广泛的分类算法之一。它不仅能进行分类，还能给出样本属于某一类别的概率。我们同样给出一个参考代码：

```
from sklearn.linear_model import LogisticRegression

# 1. 创建模型实例
lr_classifier = LogisticRegression(max_iter=1000, random_state=42)

# 2. 训练模型
print("\n--- 开始训练逻辑回归模型 ---")
lr_classifier.fit(X_train_tfidf, y_train)
print("模型训练完成！")

# 3. 进行预测
y_pred_lr = lr_classifier.predict(X_test_tfidf)
```

第四部分：分类算法实现与评估（续）

实验一 / Experiment 1

算法四：简单的多层感知机 (MLP)

多层感知机 (Multi-layer Perceptron, MLP) 是深度学习和神经网络的基础。它由一个输入层、一个或多个隐藏层和一个输出层组成。通过非线性激活函数（如ReLU），MLP可以学习数据中复杂的非线性关系。

我们将使用 sklearn 来构建一个简单的MLP模型。

```
from sklearn.neural_network import MLPClassifier

# 1. 创建模型实例
# hidden_layer_sizes=(100,) 表示一个包含 100个神经元的隐藏层
# max_iter 设置最大迭代次数以保证模型收敛
mlp_classifier = MLPClassifier(hidden_layer_sizes=(100,), max_iter=300, random_state=42)

# 2. 训练模型
print("\n--- 开始训练多层感知机 （MLP） 模型 ---")
mlp_classifier.fit(X_train_tfidf, y_train)
print("模型训练完成! ")

# 3. 进行预测
y_pred_mlp = mlp_classifier.predict(X_test_tfidf)
```

Note: 我们为大家提供了一些关键参数说明，请参考TUNING.md.

从上述算法中选择至少2项进行训练，并且比较不同方法在不同参数下的验证集结果，最后挑选最好的结果保存预测的csv文件提交。

请注意固定随机数种子，以确保实验结果可以复现。实验报告不超过5页。

2页

1. 为什么在这个数据集上，SVM和逻辑回归的表现通常很好？（提示：文本数据的稀疏性和高维性）
2. MLP模型的性能是否可以通过调整网络结构（增加层或神经元）或训练参数（增加epochs）来进一步提升？是否存在过拟合的风险？
3. 除了TF-IDF，还有哪些文本表示方法？（例如：词袋模型、Word2Vec、BERT等）

评分标准

项目	权重	评分细则
1. 代码实现（30分）		
- 模型结构合理、实现完整	10	能正确构建并运行
- 训练过程规范（含数据加载、训练循环、验证评估）	10	逻辑清晰，结构规范
- 代码规范（注释清晰、无冗余、结果可复现）	10	提供 README 运行说明
2. 模型性能（10分）		综合考虑验证集性能与模型复杂度，务必在实验报告中清晰体现
3. 实验报告（50分）		
- 报告结构完整，条理清晰	12.5	章节齐全，逻辑流畅
- 有对模型原理、实验设置的说明	12.5	参数、优化方法等说明充分
- 有结果分析、可视化图表展示	12.5	包含训练结果、参数对比与分类性能分析
- 有独立思考	12.5	不得只做概念性描述，必须结合实验现象进行论证
4. 创新（10分）		
- 特征表示、数据处理、模型改进或优化技巧等	10	体现在实验报告中，由助教酌情给分